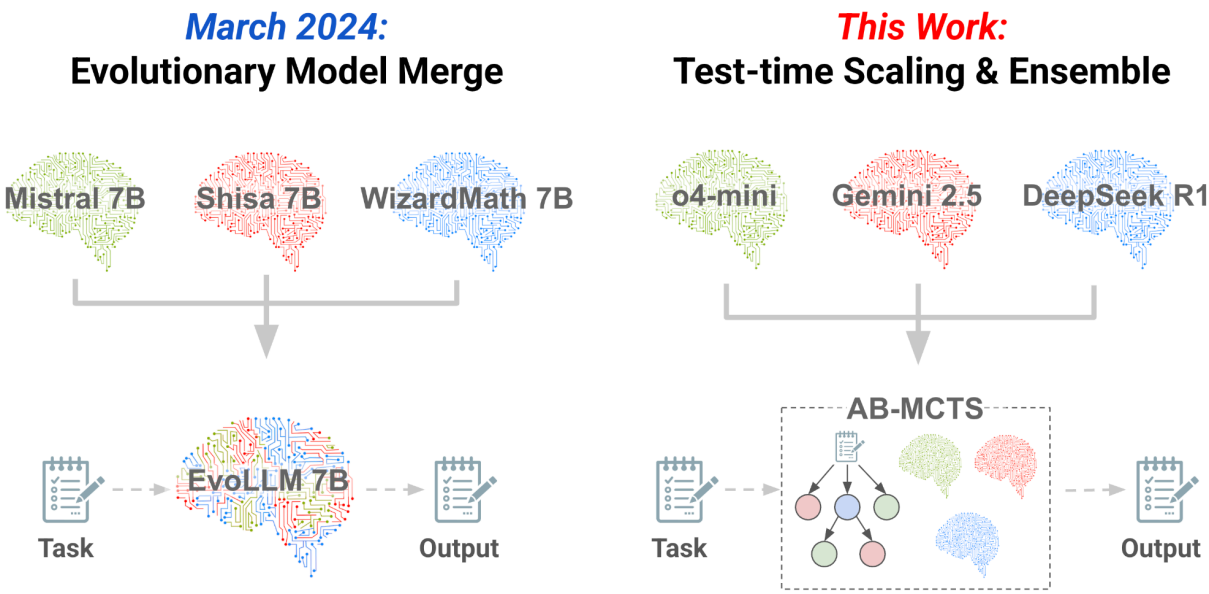
sakana.ai

Inference-Time Scaling and Collective Intelligence for Frontier AI

July 01, 2025



Following our 2024 research on evolutionary model merging, a technique for "mixing to create" better models from existing ones (left), we are now tackling the challenge of "mixing to use" frontier models (right). We have developed AB-MCTS, a new inference-time scaling algorithm that enables multiple frontier AI models to cooperate, achieving promising initial results on the ARC-AGI-2 benchmark.

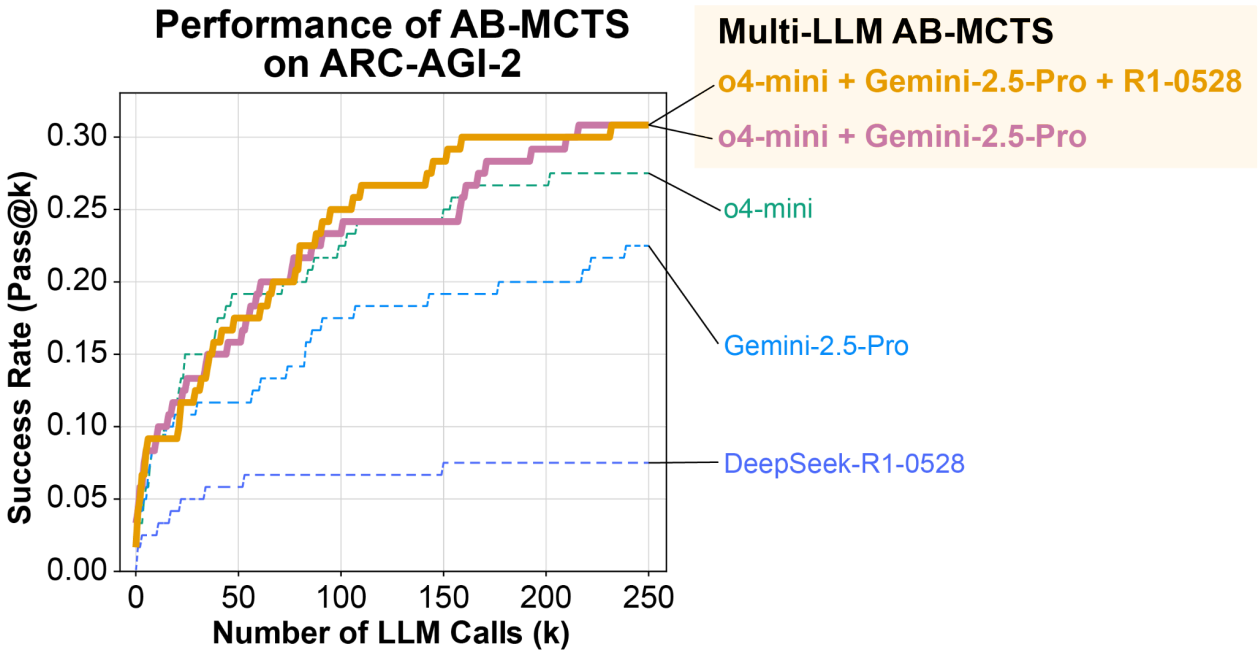
Summary

At Sakana AI, we develop AI systems by applying nature-inspired principles, such as evolution and collective intelligence. In our 2024 research on evolutionary model merging, we harnessed the vast collective intelligence of existing open-source models through evolutionary computation and model merging. This led us to a new question: *Can we utilize multiple models not only for building new models but also during inference?* Can we utilize the ever-advancing frontier models, such as ChatGPT, Gemini, and DeepSeek, to leverage them as a form of collective intelligence?

Sakana AI is proud to introduce a new inference-time scaling algorithm, **AB-MCTS (Adaptive Branching Monte Carlo Tree Search)**, that enables AI to effectively perform *trial-and-error* and allows multiple frontier AI models to *cooperate*. In this blog post, we will introduce the concept and share our

promising initial results. In particular, our AB-MCTS combination of **o4-mini** + **Gemini-2.5-Pro** + **R1-0528**, current frontier AI models as of writing, achieves strong performance on the ARC-AGI-2 benchmark, outperforming individual o4-mini, Gemini-2.5-Pro, and DeepSeek-R1-0528 models by a large margin.

- Paper: https://arxiv.org/abs/2503.04412
- Algorithm Implementation: https://github.com/SakanaAI/treequest
- ARC-AGI-2 Experiment Code: https://github.com/SakanaAI/ab-mcts-arc2



Results of AB-MCTS and Multi-LLM AB-MCTS on ARC-AGI-2, showing Pass@k as a function of the number of LLM calls.

Introduction

There is a well-known proverb: *"Two heads are better than one."* While the human brain is an extraordinary organ, humanity's greatest achievements are not the product of individual minds alone. Unprecedented feats like the Apollo program, the creation of the Internet, and the Human Genome Project were achieved not by a single genius but through the collaboration of diverse individuals and the accumulation of knowledge—in other words, *collective intelligence*. By bringing together diverse expertise and ideas, and at times clashing and fusing them, humanity has overcome countless technological barriers.

We believe the same principle holds true for AI. By pooling their intelligence, AI systems can solve problems that are insurmountable for any single model. Frontier AI models like ChatGPT, Gemini, Grok, and DeepSeek are evolving at a breathtaking pace amidst fierce competition. However, no matter how advanced they become, each model retains its own individuality stemming from its unique training data and methods. We see these biases and varied aptitudes not as limitations, but as precious resources for creating collective intelligence. Just as a dream team of diverse human experts tackles complex problems, AIs should also collaborate by bringing their unique strengths to the table.

The **AB-MCTS (Adaptive Branching Monte Carlo Tree Search)** algorithm we are introducing today is a concrete step toward realizing this vision. AB-MCTS is a method for inference-time scaling that enables frontier AIs from providers like OpenAI and DeepSeek to cooperate and efficiently perform trial-and-error. In the following sections, we will (1) introduce the concept of inference-time scaling, (2) explain how a single AI can perform trial-and-error, (3) describe our attempt to enhance this by enabling multiple AIs to cooperate, and (4) present our initial experiments using frontier models on a challenging task.

What is Inference-Time Scaling?

When faced with a difficult problem you can't solve at a glance, what do you do? Most likely, you would *think longer on your own*, engage in *hands-on trial and error,* or *collaborate with others*. Can we get AI to solve difficult problems in the same way?

A paradigm currently gaining attention is **inference-time scaling** (or test-time scaling). This is the discovery that for a single, complex problem, performance can be improved by allocating more computational resources at the time of inference. While the relationship between performance and computation during training (training-time scaling) has long been known, we now understand that a positive correlation also exists between performance and the computational budget used *after* a model is already trained. One approach is to use reinforcement learning to generate longer chains of thought, which has dramatically boosted the capabilities of so-called *reasoning models* such as OpenAI's o1/o3 and DeepSeek's R1. This corresponds to the human strategy of "thinking longer."

In addition to simply giving a reasoning model more "thinking time," we can let it revisit a problem repeatedly, refine its answers, and even start over when necessary. Imagine you are trying to solve a programming challenge. No matter how skilled a programmer you are, completing a complex program is impossible without trial-and-error—running the code, fixing bugs, and sometimes starting over from scratch. This process is not limited to programming; we humans engage in it constantly. This method is also applicable to LLMs. Our newly developed AB-MCTS is an inference-time scaling technique that enables AI to perform such *trial-and-error* efficiently and even allows different AIs to think collectively.

Three directions for inference-time scaling. Reasoning models achieve performance gains by "thinking longer" in a single attempt. In this research, we build upon this by developing AB-MCTS for trial-and-error and Multi-LLM AB-MCTS to leverage the collective intelligence of multiple frontier LLMs.

Navigating the Two Dimensions of Search: Balancing *Depth* and *Width*

Our AB-MCTS (Adaptive Branching Monte Carlo Tree Search) is an inference-time scaling method that enables LLMs to perform trial-and-error effectively. The simplest approach to making an LLM use trial-and-error is a *depth-wise* search method called Sequential Refinement. This approach uses an LLM to generate answers and then repeatedly refine them. Another method is Repeated Sampling, where LLM generates solutions from the same prompt multiple times. This is a very simple *width-wise* search that repeatedly queries the LLM, without reflecting the results of previous attempts. It leverages the

stochastic nature of LLMs, which produce different answers to the same question. While it may seem inefficient, it has been reported to outperform sequential refinement on many benchmarks.

Thus, both *searching deeper* (refining an existing solution) and *searching wider* (generating new solutions) have been proven effective for finding better answers with LLMs. However, there has been no effective way to combine them. Sequential refinement, which repeatedly refines a solution, may struggle to reach a good answer if the initial attempt is misguided. Repeated sampling, which repeatedly asks the same question, never improves upon a promising but imperfect solution. This means there was room for improvement over these simple, unidirectional methods. We believed that if we could achieve a more human-like trial-and-error process—sometimes repeating the same question to get a better initial direction, and other times refining a promising solution—we could leverage LLMs to find superior answers.

To address this, we developed AB-MCTS to flexibly search in both *depth* and *width* directions, adapting to the problem and context. With AB-MCTS, when a promising solution is found, the system can repeatedly refine it while still balancing the generation of entirely new solutions. This allows us to

obtain better answers with the same number of LLM calls compared to existing methods. In essence, AB-MCTS is a new and more effective method for inference-time scaling.

To achieve this flexible search, AB-MCTS extends Monte Carlo Tree Search (MCTS), famously used in systems like AlphaGo, and employs Thompson Sampling to decide which direction to explore. Specifically, at each node (representing either the initial prompt or a previously generated solution), AB-MCTS uses probability models to estimate the potential quality of two possible actions: generating an entirely new solution or refining an existing one. We then sample quality estimates from these models to determine the direction to pursue. A key challenge is evaluating the quality of a new, yet-to-be-generated solution. AB-MCTS effectively manages this by representing this evaluation using mixed models and probability distributions, enabling a truly flexible search. For more details on the AB-MCTS algorithm and experimental results, please refer to our paper.

AI Collaboration as a Third Search Dimension

In recent years, LLM development has exploded, with both frontier models and open-source alternatives evolving at an incredible rate. While benchmarks show that all these models are highly capable, they interestingly exhibit different characteristics. Some models excel at coding, others at creative writing, and still others at agentic, sequential task execution.

These strengths and weaknesses manifest in various ways when using LLMs to solve problems. For example, an LLM's proficiency can vary from problem to problem within the same benchmark. A model that performs poorly overall on a benchmark might excel at specific problems within it. Furthermore, some problems can be solved only when LLMs collaborate and leverage each other's strengths. For

instance, within a single problem, one LLM might be better at defining the overall strategy, while another is better at writing the specific code. Given these differing specializations, it's conceivable that by skillfully combining multiple LLMs, we can solve a wider range of problems.

To maximize the potential of multiple LLMs as a collective intelligence, we are prototyping **Multi-LLM AB-MCTS,** which adaptively explores not only the search direction but also which LLM to select for a given problem and situation. In addition to **generate a new solution** (Go Wider) and **refine an existing solution** (Go Deeper) choices in AB-MCTS, Multi-LLM AB-MCTS adds a new step: select **which LLM to use**.

Overview of the search algorithm in Multi-LLM AB-MCTS. In Step 1, it decides whether to select an existing node (search deeper) or generate a new solution from the current node (search wider). If it goes deeper, it repeats Step 1 from the next level. If it goes wider or there are no more existing nodes, it proceeds to Step 2 to select an LLM. In Step 3, the chosen LLM generates an improved solution based on the parent node, and the result is evaluated. This new solution is then added to the tree as a new node.

To make this work, we need to know which LLM is effective for each problem. However, this is unknown at the start, so the system must adjust as the search progresses. This means using a balanced mix of

LLMs in the early stages and then focusing on the ones that prove to be more promising. This can be viewed as a *multi-armed bandit problem*, a well-known problem in machine learning. However, unlike the standard multi-armed bandit problem, where the same input is presented each time, here the key is to adapt to the varying inputs, which are based on the generated answers. For LLM selection, we assigned a separate probability model to each LLM type and used Thompson sampling in a manner similar to the AB-MCTS approach described above. These probability models are updated based on each LLM's performance during the search process, so that more promising LLMs become increasingly likely to be chosen. For more detailed algorithms and implementations, please refer to our paper (Appendix D) and the code.

Experimental Results

We present initial experimental results for our in-development Multi-LLM AB-MCTS on the ARC-AGI-2 benchmark. The ARC-AGI (Abstraction and Reasoning Corpus) aims to evaluate a human-like, flexible intelligence that can efficiently reason and solve novel problems, unlike traditional metrics that test specific skills or knowledge. While ARC-AGI-1 has long been a research challenge that is easy for humans but difficult for AI, we used the even more challenging ARC-AGI-2 for our experiments.

An example problem from ARC-AGI-2. The task is to infer the common transformation rule from the three demonstration cases on the left and apply it to the test case on the right. This is one of the problems that became solvable using Multi-LLM AB-MCTS.

We investigated the potential performance of our Multi-LLM AB-MCTS on this difficult benchmark. In this experiment, we set the maximum number of search iterations (LLM calls) to 250 and instructed the models to generate the transformation rule as Python code. The search was guided by a reward reflecting how many demonstration cases were correctly solved by the generated Python code. To primarily evaluate the maximum potential performance of the search algorithm, we used the Pass@k metric, which measures whether at least one correct solution was found within k attempts. This differs from the official contest standard for ARC-AGI-2, which typically uses a Pass@2 criterion (submitting two final answers, with one being correct). A Pass@2 approach requires an additional step of selecting promising candidates from the search results. For this initial study, we focused on evaluating the "search" capability itself via Pass@k. We will briefly touch upon a tentative Pass@2 evaluation later. The experiments were performed on 120 tasks in the public evaluation set of ARC-AGI-2. For detailed experimental settings, please refer to the provided source code.

Results of AB-MCTS and Multi-LLM AB-MCTS on ARC-AGI-2, showing the Pass@250 success rate. The result of using AB-MCTS with o4-mini surpassed Repeated Sampling with o4-mini (light gray bar). Furthermore, Multi-LLM AB-MCTS, which combines Gemini-2.5-Pro and DeepSeek-R1-0528, showed an improved score at Pass@250.

Results of AB-MCTS and Multi-LLM AB-MCTS on ARC-AGI-2, showing Pass@k as a function of the number of LLM calls. The dotted lines represent the results of a control experiment using Repeated Sampling without AB-MCTS.

Repeated sampling has been considered a very effective method for ARC-AGI. Indeed, in our experiments, repeated sampling with o4-mini succeeded on 23% of the problems (i.e., generated Python code that correctly transformed the test case). This result far exceeds the single-try score and demonstrates the power of repeated sampling. However, our AB-MCTS improved the score further to 27.5%. The difference between these two methods becomes more pronounced after about 50 LLM calls.

By leveraging frontier models as a collective intelligence with Multi-LLM AB-MCTS, we were ultimately able to find correct solutions for over 30% of the problems. Interestingly, while DeepSeek-R1-0528 does not perform particularly well on its own, combining it within Multi-LLM AB-MCTS efficiently increased the number of solvable problems.

One of the key characteristics we found in Multi-LLM AB-MCTS is its ability to dynamically allocate LLMs based on their proficiency with a specific problem. This behavior is clearly illustrated in the figure below: for cases with a high success rate on the demonstration examples (left side of the figure), we observe a pronounced preference for a particular LLM. This bias occurs because the algorithm identifies

which LLM is most effective for a given problem during its search process and subsequently increases that model's usage frequency.

Proportion of LLMs used to solve each of the ARC-AGI-2 tasks at Pass@250 with Multi-LLM AB-MCTS. Results for each task are sorted by the reward value of the search tree (higher to the left). Star marks indicate successful trials.

Furthermore, we saw examples where problems that were unsolvable by any single LLM were solved by combining multiple LLMs. This went beyond simply assigning the best LLM to each problem. In the example below, even though the solution initially generated by o4-mini was incorrect, DeepSeek-R1-0528 and Gemini-2.5-Pro were able to use it as a hint to arrive at the correct solution in the next step. This demonstrates that Multi-LLM AB-MCTS can flexibly combine frontier models to solve previously unsolvable problems, pushing the limits of what is achievable by using LLMs as a collective intelligence.

An example of a search tree when solving ARC-AGI-2 with Multi-LLM AB-MCTS. The number in the node indicates the generation order, and the color represents the selected LLM. The yellow node is the one that generated code correctly transforming the test case. This is an example where no single LLM could find the solution, but a combination of LLMs succeeded.

Multi-LLM AB-MCTS enables collaboration between different LLMs. This figure shows an example where DeepSeek-R1-0528 improved upon an incorrect solution generated by o4-mini (from the problem in the figure above) to arrive at the correct answer.

As mentioned, our primary focus was on evaluating the search capability, hence the use of the Pass@k metric. For reference, when we selected two final answers using a simple rule-based method (selecting code with high reward generated later in the search), Multi-LLM AB-MCTS achieved a Pass@2 of 19.2%. While this is an excellent result for Pass@2, there is a gap of over 10% compared to the 30% Pass@k. Closing this gap by developing better final answer selection algorithms, building more sophisticated reward models, or incorporating LLM-as-a-Judge for more detailed reward design is an important area for future work. Multi-LLM AB-MCTS is an ambitious method aiming to improve performance through inference-time scaling by having multiple frontier models cooperate. In terms of combining multiple LLMs, other methods such as Multiagent Debate, Mixture-of-Agents, and LE-MCTS have also been proposed. Investigating the relationship between our method and these related technologies remains another area for future work.

Conclusion

In this work, we introduced Multi-LLM AB-MCTS as a framework for leveraging multiple high-performance frontier AIs as a collective intelligence. Our initial experiments demonstrated that Multi-LLM AB-MCTS can achieve high performance on the challenging ARC-AGI-2 benchmark. We have released the underlying algorithm as **TreeQuest**, a tree-search software framework for inference-time scaling, under the Apache 2.0 license. TreeQuest features a flexible API, allowing users to apply AB-MCTS and Multi-LLM AB-MCTS to various tasks with minimal code, and to freely implement custom scoring and generation logic. Its checkpointing feature enables easy recovery from API errors, making it practical for long-running, complex tasks.

We believe these results are significant as they suggest untapped potential in inference-time scaling. The “reasoning” models that have gained prominence since mid-2024, which optimize the inference process through reinforcement learning, have ushered in an era of inference-time scaling as the next paradigm after model scaling. Our method shows that by repeatedly executing the reasoning of these models and combining multiple LLMs with unique personalities, inference performance can be further enhanced. This points to new directions for inference-time scaling, such as *trial-and-error* and *collective intelligence* for reasoning models.

Sakana AI will continue to advance AI based on this research, focusing on the principles of evolution and collective intelligence to pioneer novel AI systems.

For more details on this research, please refer to the following materials:

- Paper: Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search https://arxiv.org/abs/2503.04412
- Algorithm Code: https://github.com/SakanaAI/treequest
- ARC-AGI-2 Experiment Code: https://github.com/SakanaAI/ab-mcts-arc2



Sakana AI

Interested in joining us? Please see our career opportunities for more information.

© Sakana AI 株式会社